Switch-AudioOutput.ps1

Master Guide — Installation, Usage, Desktop Shortcuts, Command Reference, and Full Source Code

An accessible Windows audio output switcher for the 3.5 mm jack and speakers. Designed for keyboard-only operation with JAWS and a Braille display.

Contents

1. Overview

Switch-AudioOutput.ps1 is a Windows PowerShell script that switches the computer's default audio playback device between the 3.5 mm headphone/line-out jack and the speakers. It was built specifically to be operated by a blind user running JAWS with a refreshable Braille display, so the entire tool works from the keyboard alone.

- Every action confirms itself in a plain text sentence, with no color-only signaling, no ASCII art, and no popup dialogs to navigate.
- A short confirmation tone plays after every switch: a low tone for the jack, a high tone for the speakers.
- It can be run as an interactive numbered menu, or as silent, one-line commands suitable for a Desktop shortcut with an assigned keyboard shortcut key.

2. Requirements

- Windows 10 or Windows 11.
- Windows PowerShell 5.1 or later (built into Windows).
- Internet access the *first* time the script runs, so it can install the small helper module it depends on (`AudioDeviceCmdlets`). No internet is needed after that.

3. Installation

Step 1 — Choose a folder

Create a folder to hold the tool, for example a folder named `AudioSwitcher` on the Desktop.

With the keyboard in File Explorer: press **Windows key + E** to open File Explorer, then press **Ctrl + Shift + N** to create a new folder, type `AudioSwitcher`, and press **Enter**.

Step 2 — Add the files

Place `Switch-AudioOutput.ps1` and the guide files into that folder.

Step 3 — Unblock the script file

Windows may mark a file downloaded from the internet or email as blocked. To clear this:

1. Select `Switch-AudioOutput.ps1` in File Explorer.
2. Open its context menu with **Shift + F10** (or the Menu key).
3. Choose **Properties** and press Enter.
4. Tab to the **Unblock** checkbox, if present, and press **Space** to check it.
5. Tab to **OK** and press Enter.

If no Unblock checkbox appears, the file was not blocked.

Step 4 — First run

Press **Windows key + R**, type `powershell`, and press Enter to open a PowerShell window. Then type (using your own folder path):

```
powershell -ExecutionPolicy Bypass -File
"C:\Users\YourName\Desktop\AudioSwitcher\Switch-AudioOutput.ps1" -
ListDevices
```

The first run installs the AudioDeviceCmdlets module automatically — wait for "Module installed successfully." This happens once. The script then lists your playback devices, for example:

```
Playback devices known to Windows:
1. Speakers (Realtek Audio) (currently default)
2. Headphones (Realtek Audio)
```

Note the exact device names for the jack and the speakers.

Step 5 — Configure device names, if needed

The defaults are "Headphones" (jack) and "Speakers" (speakers). If your device names differ, set them once:

```
powershell -ExecutionPolicy Bypass -File
"C:\Users\YourName\Desktop\AudioSwitcher\Switch-AudioOutput.ps1" -
JackDeviceName "Headphones" -SpeakerDeviceName "Speakers" -Status
```

These names are saved automatically to a small config file next to the script, so they are only typed once.

4. Everyday Use

Option A — Interactive menu

```
powershell -ExecutionPolicy Bypass -File
"C:\Users\YourName\Desktop\AudioSwitcher\Switch-AudioOutput.ps1"
```

This displays and reads:

```
Audio Output Switcher
1. Switch to 3.5 mm jack (Headphones)
2. Switch to speakers (Speakers)
3. Toggle between the two
4. Announce current output device
5. List all playback devices
6. Set which device names to use for jack and speakers
Q. Quit
```

Type a number or letter and press Enter. Every choice confirms itself in text plus a tone, for example: "Audio output switched to: Headphones (Realtek Audio)." Type **Q** and press Enter to quit.

Option B — Direct one-line commands (no menu, no prompts)

Command	What it does
`-Jack`	Switches to the 3.5 mm jack device and exits.
`-Speakers`	Switches to the speaker device and exits.
`-Toggle`	Switches to whichever device isn't currently active, and exits.
`-Status`	Announces the current default device and exits.

Example:

```
powershell -ExecutionPolicy Bypass -File
"C:\Users\YourName\Desktop\AudioSwitcher\Switch-AudioOutput.ps1" -Toggle
```

5. Complete Command Reference

All parameters accepted by Switch-AudioOutput.ps1. Any combination of `-JackDeviceName`, `-SpeakerDeviceName`, and `-NoBeep` can be added to any of the action parameters below.

Parameter	Type	Description
`-Jack`	switch	Switch output to the 3.5 mm jack device and exit immediately. No menu.
`-Speakers`	switch	Switch output to the speaker device and exit immediately. No menu.
`-Toggle`	switch	Switch to whichever of the two configured devices is NOT currently active, and exit immediately. No menu. Intended for binding to a Desktop shortcut keyboard shortcut for single-key operation.
`-Status`	switch	Announce the current default playback device and exit.
`-ListDevices`	switch	List every playback device Windows knows about, with its exact name, for use with -JackDeviceName / -SpeakerDeviceName.
`-JackDeviceName`	string	Text found in the jack device's name (case-insensitive substring match, e.g. "Headphones"). Saved to the config file after first use.
`-SpeakerDeviceName`	string	Text found in the speaker device's name (matched the same way, e.g. "Speakers"). Saved to the config file after first use.
`-NoBeep`	switch	Suppress the confirmation beep. Text feedback is always given regardless of this switch.

(no parameters)	—	Opens the interactive numbered menu described in Section 4.

Interactive menu options

Key	Action
1	Switch to the 3.5 mm jack
2	Switch to the speakers
3	Toggle between the two
4	Announce the current output device
5	List all playback devices
6	Set which device names to use for jack and speakers
Q	Quit

All example invocations

```
.\Switch-AudioOutput.ps1                # opens the accessible numbered
menu
.\Switch-AudioOutput.ps1 -Jack          # switch to the 3.5 mm jack, no
prompts
.\Switch-AudioOutput.ps1 -Speakers      # switch to the speakers, no
prompts
.\Switch-AudioOutput.ps1 -Toggle        # flip between jack and speakers
.\Switch-AudioOutput.ps1 -Status        # announce the current device
.\Switch-AudioOutput.ps1 -ListDevices   # list every playback device by
exact name
.\Switch-AudioOutput.ps1 -JackDeviceName "Headphones" -SpeakerDeviceName
"Speakers"
.\Switch-AudioOutput.ps1 -Toggle -NoBeep # toggle with text feedback only,
no tone
```

6. Building a Desktop Shortcut

A Desktop shortcut with an assigned keyboard shortcut key lets you switch audio output with a single keystroke from anywhere on the desktop — no PowerShell window or menu needed to open first.

Choosing which shortcut(s) to build

- **Toggle shortcut** — one key flips between jack and speakers.
- **Jack shortcut** — one key always selects the jack.
- **Speakers shortcut** — one key always selects the speakers.

Many people create separate Jack and Speakers shortcuts so each key always does the same thing, rather than relying on Toggle's current-state awareness.

Step by step

1. Open the folder containing the script (**Windows key + E**, then navigate there).
2. Open the context menu with **Shift + F10**, arrow to **New**, press Enter or Right Arrow, then arrow to **Shortcut** and press Enter.
3. Type the command line for the shortcut you want, then press Enter:

   ```
   powershell -ExecutionPolicy Bypass -File
   "C:\Users\YourName\Desktop\AudioSwitcher\Switch-AudioOutput.ps1" -
   Toggle
   powershell -ExecutionPolicy Bypass -File
   "C:\Users\YourName\Desktop\AudioSwitcher\Switch-AudioOutput.ps1" -
   Jack
   powershell -ExecutionPolicy Bypass -File
   "C:\Users\YourName\Desktop\AudioSwitcher\Switch-AudioOutput.ps1" -
   Speakers
   ```

4. Give the shortcut a clear name (for example "Switch Audio To Jack") and finish.
5. Select the new shortcut, open its Properties (**Shift + F10** → Properties).

6. Tab to the **Shortcut key** field and press a single letter, e.g. **J**. Windows fills in **Ctrl + Alt + J** automatically.
7. Tab to **OK** and press Enter to save.

Test it by pressing the assigned key combination from anywhere on the desktop. You should hear the confirmation tone and, if a console window is visible, the confirmation sentence.

> **Tip:** Repeat the steps above to create more than one shortcut. Give each a different key (for example Ctrl+Alt+J for jack, Ctrl+Alt+S for speakers) so they don't conflict. Shortcuts can be moved anywhere afterward (for example onto the Desktop itself with Ctrl+X then Ctrl+V) — the assigned key keeps working regardless of the shortcut file's location.

7. Troubleshooting

Symptom	What to do
"Could not find a playback device matching..."	Run `-ListDevices` again, note the exact device name, and re-run with `-JackDeviceName` or `-SpeakerDeviceName` set to that exact text.
"Automatic install failed" during first run	Usually means no internet connection at that moment. Reconnect and try again. As a fallback, run this once in PowerShell: `Install-Module -Name AudioDeviceCmdlets -Scope CurrentUser -Force`
PowerShell mentions "execution policy" and refuses to run	Make sure the command starts with exactly `powershell -ExecutionPolicy Bypass -File` as shown throughout this guide.
Nothing seems to happen after switching	Run `-Status` to confirm the device Windows currently believes is active. Some devices take a second or two to switch audibly.
Shortcut key field won't accept a letter	Tab directly into the Shortcut key field before pressing the key. Keys like Escape and Tab cannot be used as the shortcut key.
Pressing the assigned key combination does nothing	Reopen the shortcut's Properties and check the Target field for typos, especially the quotation marks around the file path.
Two shortcuts share the same key combination	Only one responds. Open Properties on each and assign a different letter.

8. Full Source Code Listing

The complete, unmodified contents of `Switch-AudioOutput.ps1`.

```
#Requires -Version 5.1
<#
.SYNOPSIS
    Switches the default Windows audio output between the 3.5 mm jack and
    the speakers, with a fully keyboard-driven, screen-reader-friendly
    interface.

.DESCRIPTION
    Built to be operated entirely by keyboard. Every state change is written
    as a plain text sentence (no ASCII art, no color-only signals, no popup
    dialogs), so JAWS and a refreshable Braille display both announce it
    correctly as it appears in the console.

    Uses the "AudioDeviceCmdlets" PowerShell module to talk to the same
    device list found in Settings > System > Sound. The module is installed
    automatically, for the current user only, the first time this script
    runs (this needs internet access once).

.PARAMETER Jack
    Switch output to the 3.5 mm jack device and exit immediately. No menu.

.PARAMETER Speakers
    Switch output to the speaker device and exit immediately. No menu.

.PARAMETER Toggle
    Switch to whichever of the two configured devices is NOT currently
    active, and exit immediately. No menu. This is the one to bind to a
    keyboard shortcut for single-key operation (see windows/README.md).

.PARAMETER Status
    Announce the current default playback device and exit.

.PARAMETER ListDevices
    List every playback device Windows currently knows about, with its
    exact name, so you can copy the exact text into -JackDeviceName /
    -SpeakerDeviceName.

.PARAMETER JackDeviceName
    Text found in the jack device's name (matched case-insensitively as a
    substring, e.g. "Headphones"). Saved to a config file after first use,
```

```
    so you only need to type it once.

.PARAMETER SpeakerDeviceName
    Text found in the speaker device's name (matched the same way, e.g.
    "Speakers"). Saved to a config file after first use.

.PARAMETER NoBeep
    Suppress the confirmation beep. Text feedback is always given
    regardless of this switch.

.EXAMPLE
    .\Switch-AudioOutput.ps1
    Opens the accessible numbered menu.

.EXAMPLE
    .\Switch-AudioOutput.ps1 -Toggle
    Flips output between jack and speakers with no prompts.

.EXAMPLE
    .\Switch-AudioOutput.ps1 -ListDevices
    Prints every playback device name so you can find the exact wording
    Windows uses for your jack / speakers before configuring the script.
#>

[CmdletBinding(DefaultParameterSetName = 'Menu')]
param(
    [Parameter(ParameterSetName = 'Jack')]
    [switch]$Jack,

    [Parameter(ParameterSetName = 'Speakers')]
    [switch]$Speakers,

    [Parameter(ParameterSetName = 'Toggle')]
    [switch]$Toggle,

    [Parameter(ParameterSetName = 'Status')]
    [switch]$Status,

    [Parameter(ParameterSetName = 'ListDevices')]
    [switch]$ListDevices,

    [string]$JackDeviceName,
    [string]$SpeakerDeviceName,

    [switch]$NoBeep
)

$ErrorActionPreference = 'Stop'
$ConfigPath = Join-Path $PSScriptRoot 'AudioSwitcher.config.json'
```

```
# Distinct confirmation tones so the choice is audible even without reading
# the text - a low tone for the jack, a high tone for the speakers.
$JackTone = 500
$SpeakerTone = 900

function Say {
    # Single choke point for all user-facing text: plain sentences only,
    # so JAWS and a Braille display read every line correctly.
    param([string]$Text)
    Write-Host $Text
}

function Confirm-Beep {
    param([int]$Frequency = 700, [int]$DurationMs = 150)
    if (-not $NoBeep) {
        try { [console]::beep($Frequency, $DurationMs) } catch { }
    }
}

function Confirm-AudioModule {
    if (Get-Module -ListAvailable -Name AudioDeviceCmdlets) {
        return
    }
    Say "The AudioDeviceCmdlets module is required and is not installed yet."
    Say "Installing it now for the current user. This only happens once."
    try {
        try {
            Set-PSRepository -Name PSGallery -InstallationPolicy Trusted -ErrorAction
Stop
        } catch { }
        Install-Module -Name AudioDeviceCmdlets -Scope CurrentUser -Force -
AllowClobber -Confirm:$false
        Say "Module installed successfully."
    } catch {
        Say "Automatic install failed: $($_.Exception.Message)"
        Say "Open PowerShell yourself and run this command, then re-run this script:"
        Say "    Install-Module -Name AudioDeviceCmdlets -Scope CurrentUser -Force"
        exit 1
    }
}

function Get-SavedConfig {
    if (Test-Path $ConfigPath) {
        try { return Get-Content $ConfigPath -Raw | ConvertFrom-Json } catch { return
$null }
    }
    return $null
}
```

```
function Save-Config {
    param([string]$JackName, [string]$SpeakerName)
    [pscustomobject]@{
        JackDeviceName    = $JackName
        SpeakerDeviceName = $SpeakerName
    } | ConvertTo-Json | Set-Content -Path $ConfigPath -Encoding UTF8
}

function Get-PlaybackDevices {
    Get-AudioDevice -List | Where-Object { $_.Type -eq 'Playback' }
}

function Find-Device {
    param([string]$NameFragment)
    Get-PlaybackDevices | Where-Object { $_.Name -like "*$NameFragment*" } | Select-
Object -First 1
}

function Get-CurrentDevice {
    Get-AudioDevice -Playback
}

function Set-OutputDevice {
    param($Device, [string]$FriendlyLabel, [int]$Frequency = 700)
    if (-not $Device) {
        Say "Could not find a playback device matching '$FriendlyLabel'."
        Say "Run this script with -ListDevices to see the exact device names Windows
has, then re-run with -JackDeviceName or -SpeakerDeviceName set to that exact text."
        return $false
    }
    Set-AudioDevice -Index $Device.Index | Out-Null
    Say "Audio output switched to: $($Device.Name)"
    Confirm-Beep -Frequency $Frequency
    return $true
}

function Resolve-DeviceNames {
    # Command-line parameters win, then a saved config file, then defaults.
    $config = Get-SavedConfig
    $jackName = if ($JackDeviceName) { $JackDeviceName }
                elseif ($config -and $config.JackDeviceName) { $config.JackDeviceName
}
                else { 'Headphones' }
    $speakerName = if ($SpeakerDeviceName) { $SpeakerDeviceName }
                   elseif ($config -and $config.SpeakerDeviceName) {
$config.SpeakerDeviceName }
                   else { 'Speakers' }
```

```
    if ($JackDeviceName -or $SpeakerDeviceName) {
        Save-Config -JackName $jackName -SpeakerName $speakerName
    }
    return @{ Jack = $jackName; Speaker = $speakerName }
}

function Show-Status {
    $current = Get-CurrentDevice
    if ($current) {
        Say "Current default audio output: $($current.Name)"
    } else {
        Say "Could not determine the current audio output device."
    }
}

function Show-DeviceList {
    $devices = Get-PlaybackDevices
    if (-not $devices) {
        Say "No playback devices were found."
        return
    }
    Say "Playback devices known to Windows:"
    $i = 1
    foreach ($d in $devices) {
        $marker = if ($d.Default) { ' (currently default)' } else { '' }
        Say "$i. $($d.Name)$marker"
        $i++
    }
}

function Invoke-Toggle {
    param([hashtable]$Names)
    $current = Get-CurrentDevice
    $jackDevice = Find-Device -NameFragment $Names.Jack
    $speakerDevice = Find-Device -NameFragment $Names.Speaker

    if (-not $jackDevice -or -not $speakerDevice) {
        Say "Cannot toggle: one or both configured devices were not found."
        Show-DeviceList
        return
    }

    if ($current -and $current.Index -eq $jackDevice.Index) {
        Set-OutputDevice -Device $speakerDevice -FriendlyLabel $Names.Speaker -
Frequency $SpeakerTone
    } else {
        Set-OutputDevice -Device $jackDevice -FriendlyLabel $Names.Jack -Frequency
$JackTone
    }
```

```
}

function Set-DeviceNamesInteractive {
    Say "Enter text found in the jack device's name, for example Headphones."
    $jackInput = Read-Host "Jack device name text"
    Say "Enter text found in the speaker device's name, for example Speakers."
    $speakerInput = Read-Host "Speaker device name text"
    $names = Resolve-DeviceNames
    if ($jackInput) { $names.Jack = $jackInput }
    if ($speakerInput) { $names.Speaker = $speakerInput }
    Save-Config -JackName $names.Jack -SpeakerName $names.Speaker
    Say "Saved. These names will be used automatically next time."
    return $names
}

function Show-Menu {
    param([hashtable]$Names)
    Say ""
    Say "Audio Output Switcher"
    Say "1. Switch to 3.5 mm jack ($($Names.Jack))"
    Say "2. Switch to speakers ($($Names.Speaker))"
    Say "3. Toggle between the two"
    Say "4. Announce current output device"
    Say "5. List all playback devices"
    Say "6. Set which device names to use for jack and speakers"
    Say "Q. Quit"
    Say ""
    (Read-Host "Type a number or letter, then press Enter").Trim()
}

# ---- Entry point ----

Confirm-AudioModule

switch ($PSCmdlet.ParameterSetName) {
    'ListDevices' {
        Show-DeviceList
    }
    'Status' {
        Show-Status
    }
    'Jack' {
        $names = Resolve-DeviceNames
        Set-OutputDevice -Device (Find-Device -NameFragment $names.Jack) -
FriendlyLabel $names.Jack -Frequency $JackTone | Out-Null
    }
    'Speakers' {
        $names = Resolve-DeviceNames
        Set-OutputDevice -Device (Find-Device -NameFragment $names.Speaker) -
```

```
FriendlyLabel $names.Speaker -Frequency $SpeakerTone | Out-Null
    }
    'Toggle' {
        $names = Resolve-DeviceNames
        Invoke-Toggle -Names $names
    }
    default {
        $names = Resolve-DeviceNames
        Say "Welcome to the Audio Output Switcher."
        Say "This menu is fully keyboard-driven. Type a choice and press Enter."
        $running = $true
        while ($running) {
            $choice = (Show-Menu -Names $names).ToUpper()
            switch ($choice) {
                '1' { Set-OutputDevice -Device (Find-Device -NameFragment $names.Jack)
-FriendlyLabel $names.Jack -Frequency $JackTone | Out-Null }
                '2' { Set-OutputDevice -Device (Find-Device -NameFragment
$names.Speaker) -FriendlyLabel $names.Speaker -Frequency $SpeakerTone | Out-Null }
                '3' { Invoke-Toggle -Names $names }
                '4' { Show-Status }
                '5' { Show-DeviceList }
                '6' { $names = Set-DeviceNamesInteractive }
                'Q' { Say "Goodbye."; $running = $false }
                default { Say "Not a valid choice. Type 1, 2, 3, 4, 5, 6, or Q." }
            }
        }
    }
}
```